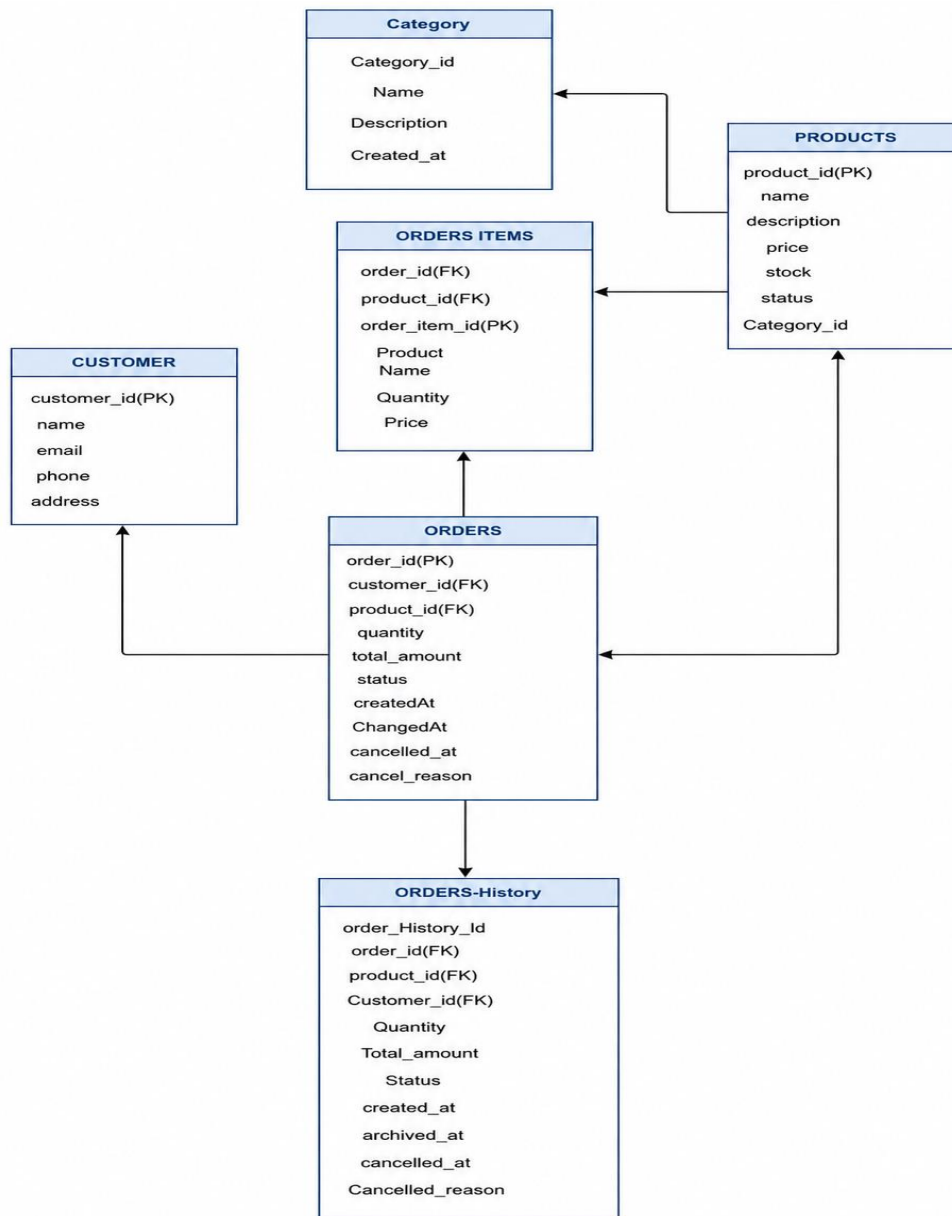
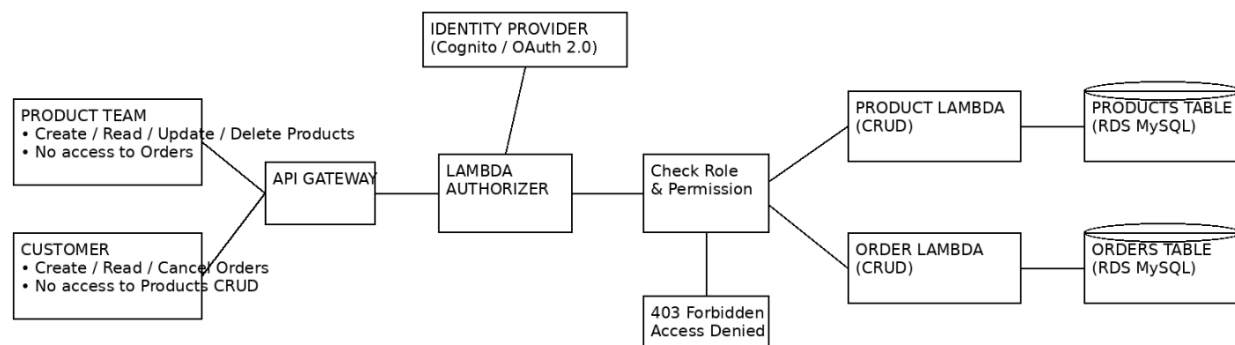


DATABASE SCHEMA



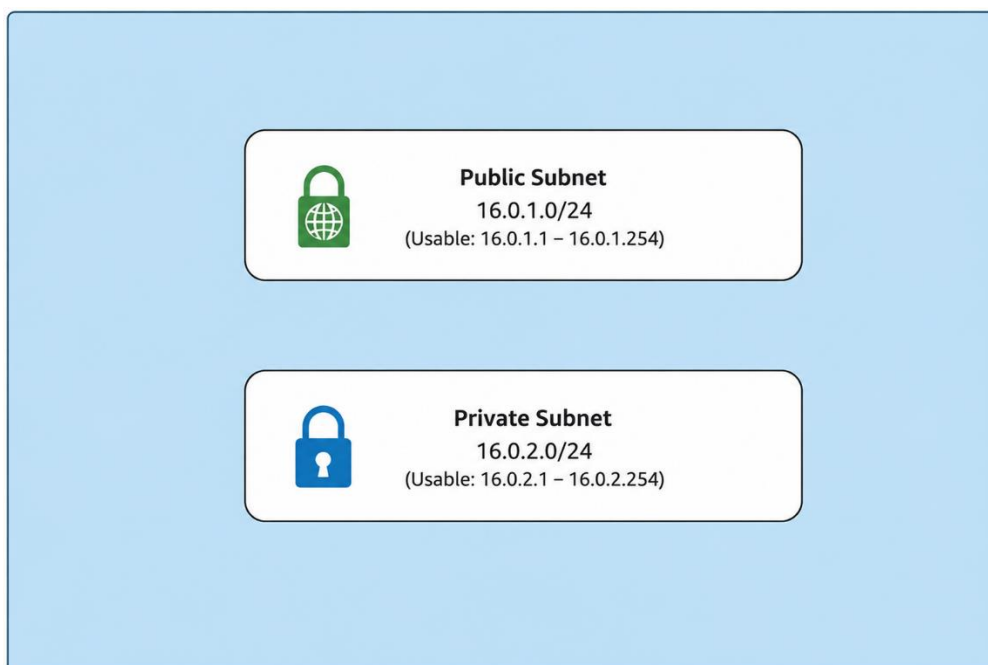
1. **Customer:** Stores customer details such as name, email, phone, and address.
2. **Category:** Stores product categories and their descriptions.
3. **Products:** Stores product details, price, stock, and category reference.
4. **Orders:** Stores active order details, including customer, product, quantity, amount, and status.
5. **Comments:** Allows customers to comment/rate products or orders.
6. **Order History:** Preserves cancelled/deleted orders before they are removed from ORDERS.
7. **Audit Logs:** Records who changed what in the PRODUCTS table, including old and new values.
8. **Relationships:** Customers and products are linked to orders through foreign keys, while products belong to categories.

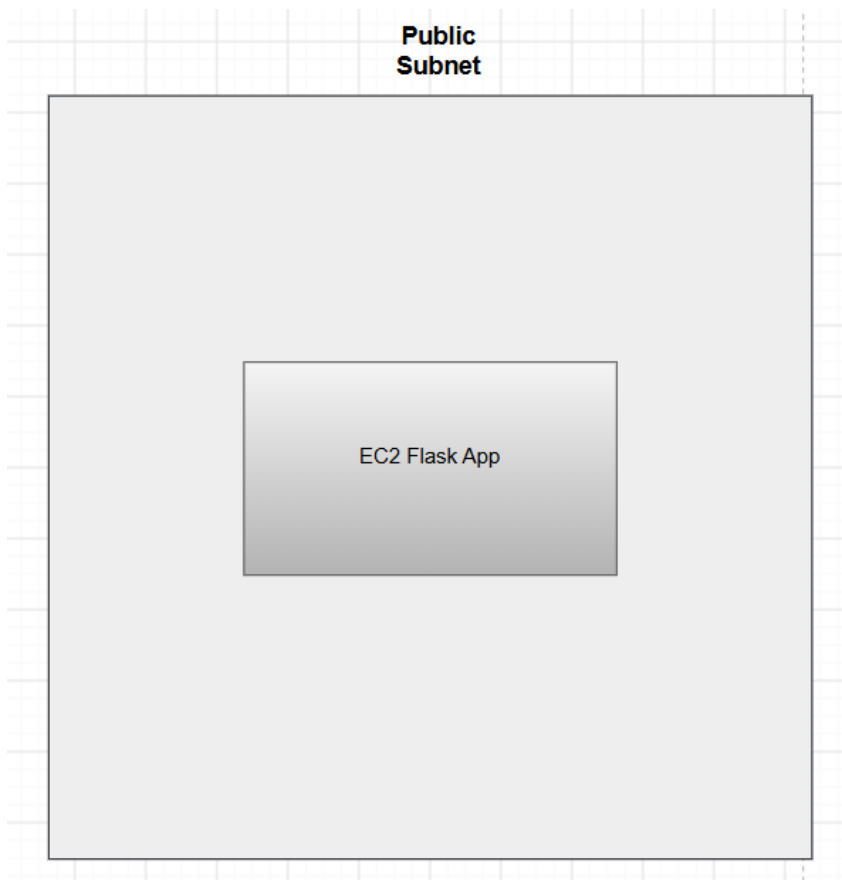
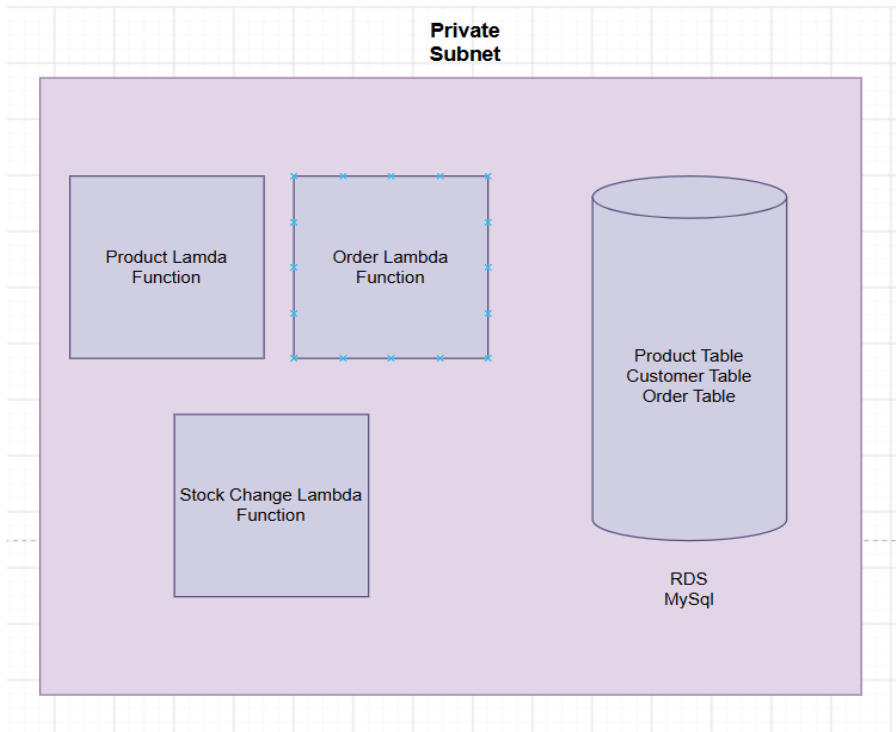
API Request Authorization Flow



1. **Authenticate:** Customer/Product Team logs in and receives a JWT containing their role.
2. **Send Request:** User sends the JWT with the API request to API Gateway.
3. **Authorize:** Lambda Authorizer validates the token and identifies the user's role.
4. **Check Permissions:** Authorizer allows only:
 5. **product_team** → Product endpoints / Product table CRUD
 6. **customer** → Order endpoints / Order table operations
7. **Execute:** Authorized request reaches the respective Lambda; unauthorized requests return **403 Forbidden**.

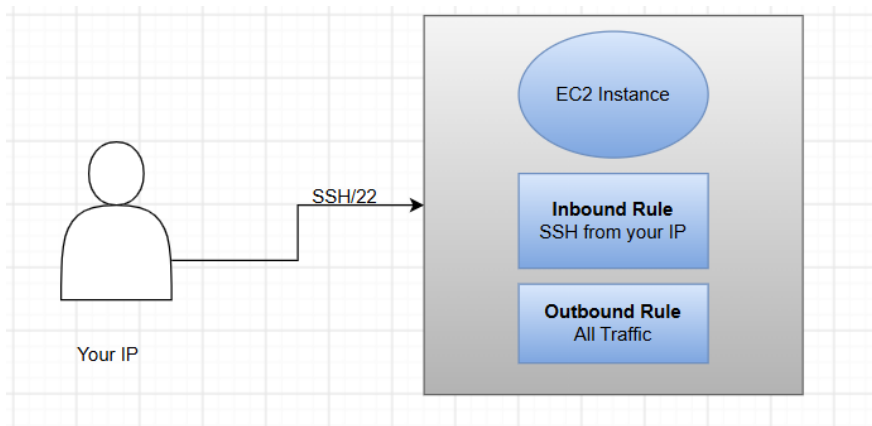
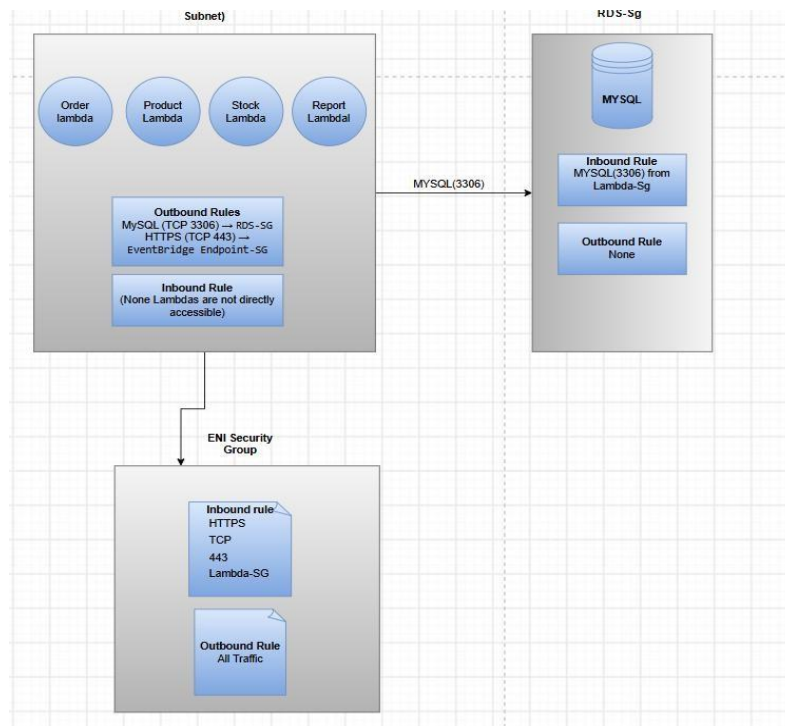
Cloud Mart VPC





1. VPC Architecture: Network segmented into Public Subnet (16.0.1.0/24) and Private Subnet (16.0.2.0/24) for security and resource isolation.
2. Application Layer: Flask application hosted on an EC2 instance in the Public Subnet serves client requests.
3. Business Logic: Product, Order, and Stock Change Lambda functions in the Private Subnet handle backend processing.
4. Database Layer: Amazon RDS MySQL stores Product, Customer, and Order data securely within the Private Subnet.
5. Secure Data Flow: EC2 communicates with Lambda functions, which access RDS, ensuring the database is never directly exposed to the internet.

Security Groups



1. Lambda Security Group: Product, Order, Stock, and Report Lambda functions run in private subnets and are not directly accessible externally.
2. Controlled Database Access: Lambda functions connect to RDS MySQL only through TCP 3306.
3. Restricted RDS Access: RDS allows inbound TCP 3306 exclusively from the Lambda Security Group.
4. Private AWS Service Access: Lambda functions use HTTPS (443) through VPC endpoints to securely access supported AWS services such as EventBridge.
5. Secure EC2 Administration: EC2 permits SSH (22) only from the authorized user IP and allows required outbound traffic.

IAM ROLES

1. CloudMartGitHubActionsRole

Usedby: GitHubActions

Purpose: Deploy infrastructure

Permissions:

- **CloudFormation** — Full Access
- **IAM** — Required permissions
- **EC2** — Required permissions
- **RDS** — Required permissions
- **S3** — Required permissions
- **SNS** — Required permissions
- **EventBridge** — Required permissions
- **SSM Parameter Store** — Read
- **CloudWatch Logs**

2. CloudMartOrderLambdaRole

Usedby: OrderLambda

Purpose: Handles all order APIs

Permissions:

- **RDS Data Access** — MySQL
- **SSM Parameter Store** — Read
- **CloudWatch Logs**
- **Invoke Check Stock Lambda** — if required

3. CloudMartProductLambdaRole

Usedby:ProductLambda

Purpose: Read products

Permissions:

- **RDS Data Access** — MySQL (Read Only)
- **SSM Parameter Store** — Read
- **CloudWatch Logs**

4. CloudMartCheckStockLambdaRole

Usedby:CheckStockLambda

Purpose: Checks stock and publishes events

Permissions:

- **RDS Data Access** — MySQL (Read Only)
- **EventBridge** — PutEvents
- **SSM Parameter Store** — Read
- **CloudWatch Logs**

5. CloudMartReportLambdaRole

Usedby:ReportLambda

Purpose: Generates reports

Permissions:

- **RDS Data Access** — MySQL (Read Only)
- **S3** — PutObject
- **SSM Parameter Store** — Read
- **CloudWatch Log**

6. CloudMartAuthorizerRole

Usedby:LambdaAuthorizer

Purpose: API Gateway authentication/authorization

Permissions:

- **SSM Parameter Store** — Read
- **CloudWatch Logs**

Architecture and Technical Decisions

1. Database — RDS MySQL

Decision: Use Amazon RDS MySQL as the primary relational database for all application tables.

Why: The application contains related data such as customers, products, and orders. RDS MySQL provides relational relationships, SQL queries, and transaction support.

Advantages:

- Supports relational data and foreign keys
- Provides transaction support
- Managed database service
- Easy to query using SQL

Trade-off: Database connections and scaling need to be managed carefully, especially with Lambda.

2. Order Processing — Single Order Lambda

Decision: Use a single Order Lambda for the order-related business logic.

It handles operations such as:

- Creating a new order
- Checking product availability before placing an order
- Retrieving customer orders
- Retrieving product information

Why: Keeping the order-related logic in one Lambda keeps the design simple and makes the order workflow easier to manage.

Advantages:

- Simple architecture
- Centralized order logic
- Fewer Lambda functions to manage

Trade-off: The function can become larger as the application grows.

3. Synchronous Order Processing

Decision: Process order creation synchronously.

Flow:

Client → API Gateway → Authorizer → Order Lambda → RDS

Why: The customer needs an immediate response indicating whether the order was successfully placed or rejected due to insufficient stock.

Advantages:

- Immediate response
- Simple request/response model
- Order is confirmed only after the required database operations complete

Trade-off: The API request remains active while the order is being processed.

4. Asynchronous Stock Monitoring

Decision: Perform the post-order stock check separately from the main order transaction.

Flow:

Order Lambda → Check Stock Lambda → EventBridge → SNS

Why: Checking whether stock has fallen below the threshold is not required to determine whether the customer's order was successfully placed.

Advantages:

- Keeps Order Lambda focused on order processing
- Separates notification logic
- Failure in notification processing does not need to invalidate the completed order

Trade-off: Stock alerts are processed asynchronously.

5. EventBridge

Decision: Use EventBridge to route stock-threshold events.

Flow:

Check Stock Lambda → EventBridge → SNS

Why: EventBridge decouples the stock-checking logic from the notification mechanism and allows additional targets to be added later.

Advantages:

- Loose coupling
- Event-based architecture
- Easy to add additional consumers

Trade-off: Adds another AWS service to the architecture.

6. VPC and Network Security

Decision: Keep Lambda and RDS in private subnets.

Flow:

Private Lambda → RDS

Why: The database should not be directly accessible from the public internet.

Security Groups:

Lambda-SG → RDS-SG : TCP 3306

Only Lambda resources with the appropriate Security Group can connect to RDS.

Advantages:

- RDS remains private
- Network isolation
- Controlled database access

Trade-off: Private-subnet networking requires additional configuration for Lambda access to AWS services.

8. IAM Roles

Decision: Use separate IAM roles for different Lambda functions.

For example:

Role	Purpose
CloudMartOrderLambdaRole	Order processing and RDS access
CloudMartProductLambdaRole	Product operations and RDS read access
CloudMartCheckStockLambdaRole	Stock checking and EventBridge publishing
CloudMartReportLambdaRole	Report generation and S3 access
CloudMartAuthorizerRole	Authentication/authorization
CloudMartGitHubActionsRole	Infrastructure deployment

Why: Each component receives only the permissions it actually needs.

Advantages:

- Least-privilege access
- Better security
- Reduced impact if a component is compromised

Trade-off: More IAM roles need to be managed.

9. GitHub Actions — OIDC

Decision: Use GitHub OIDC for AWS authentication.

Flow:

GitHub Actions → OIDC → CloudMartGitHubActionsRole → AWS

Why: It avoids storing long-lived AWS access keys in GitHub.

Advantages:

- No permanent AWS credentials in GitHub
- Temporary credentials
- Repository/branch can be restricted through the IAM trust policy

Trade-off: Requires initial OIDC and IAM configuration.

10. Monitoring — CloudWatch

Decision: Use CloudWatch for application and infrastructure monitoring.

Monitored components include:

- Lambda
- API Gateway
- RDS
- Application logs
- Errors and failures

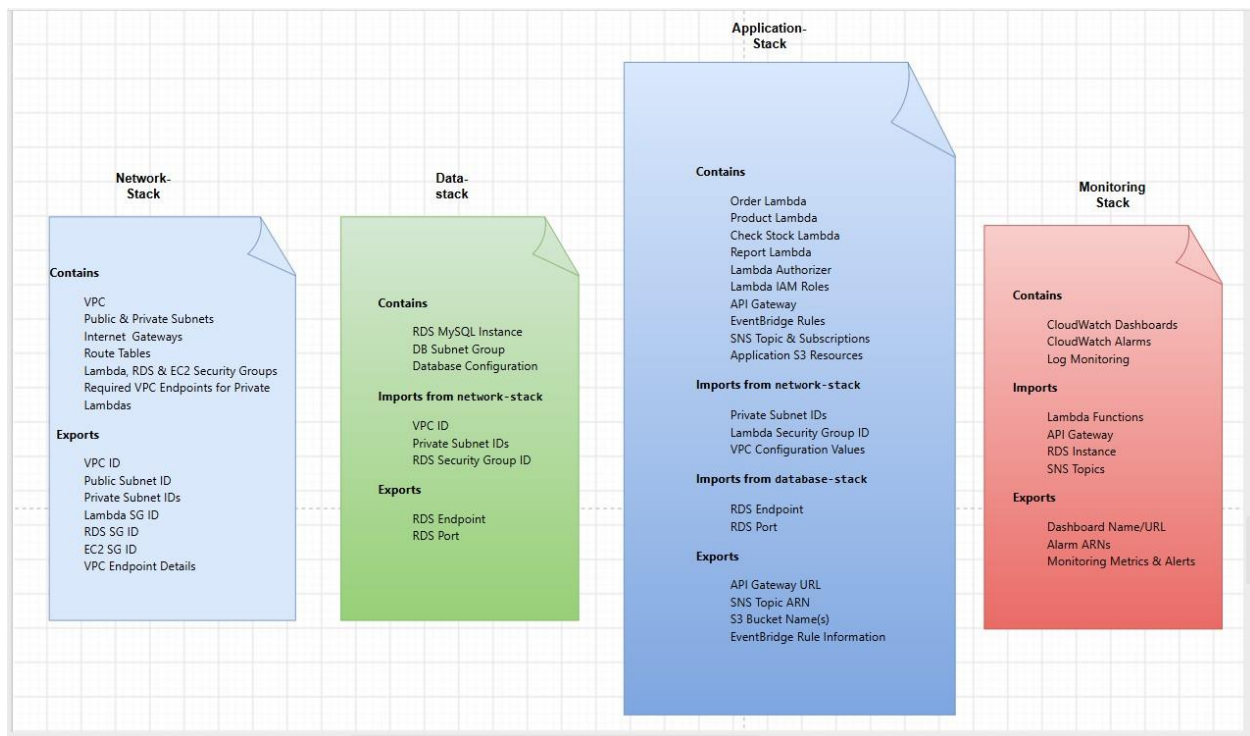
Why: Operations need a centralized view of system health and failures.

Advantages:

- Centralized logging
- Metrics and alarms
- Easier troubleshooting

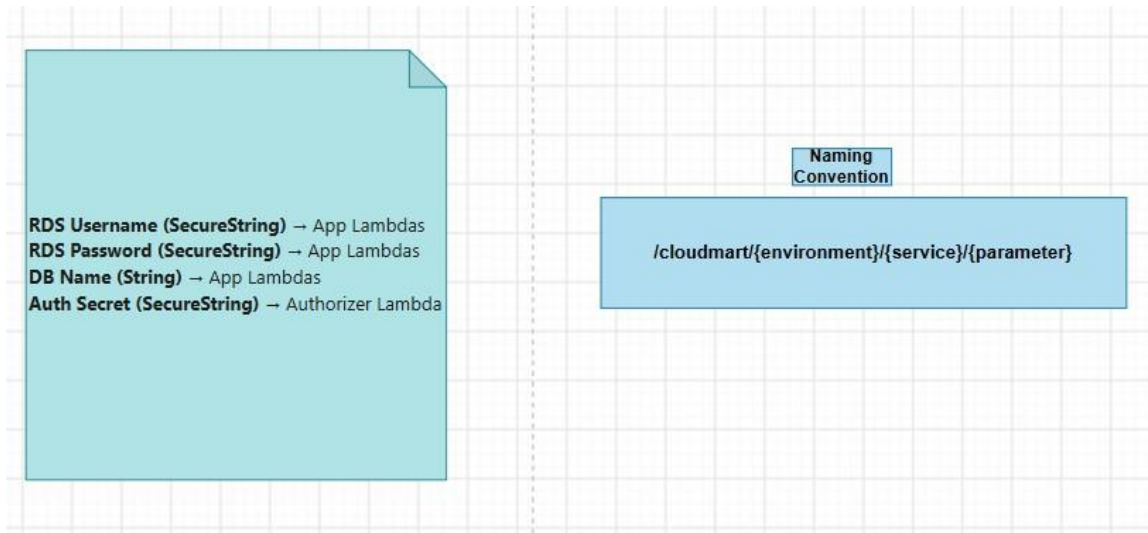
Trade-off: Dashboards, metrics, and alarms require configuration.

CloudFormation Stack Design



1. **Network Stack** establishes the secure networking foundation, including VPC, subnets, gateways, route tables, security groups, and required VPC endpoints.
2. **Database Stack** deploys the RDS MySQL database in private subnets using resources imported from the Network Stack.
3. **Application Stack** hosts serverless business services (Lambda, API Gateway, SNS, EventBridge, and S3) and connects securely to the database.
4. **Monitoring Stack** provides centralized observability through CloudWatch dashboards, alarms, logs, and performance monitoring.
5. **Cross-stack exports and imports** enable secure resource sharing, promoting modularity, scalability, and independent stack deployments.

Configuration Structure



1. SSM stores only configuration and secrets, not application data.
2. RDS username and password are stored as SecureString.
3. DB name is stored as a String.
4. Auth token is stored as SecureString and used by the Authorizer Lambda.
5. Consistent naming: `/cloudmart/{environment}/{service}/{parameter}` for easy management across environments.

Environment Parameterization

- 1) Use both CloudFormation and SSM Parameters.
- 2) CloudFormation Parameters → define the deployment environment (dev / prod).
- 3) SSM Parameter Store → store environment-specific configuration and secrets.
- 4) Naming convention: `/cloudmart/{environment}/{service}/{parameter}`.
- 5) Same convention across all stacks to keep deployments consistent.

API Request and Response Contracts

1. Product APIs					
Method	Endpoint	Function	Purpose	Access	
GET	/products	Product Lambda	Get all products	Customer, Product Team	
GET	/products/{id}	Product Lambda	Get a specific product	Customer, Product Team	
POST	/products	Product Lambda	Add a new product	Product Team	
PATCH	/products/{id}	Product Lambda	Update selected product fields	Product Team	
DELETE	/products/{id}	Product Lambda	Delete a product	Product Team	

3. Order APIs					
Method	Endpoint	Function	Purpose	Access	
POST	/orders	Order Lambda	Place a new order	Customer	
GET	/orders	Order Lambda	Get customer's orders	Customer	
GET	/orders/{id}	Order Lambda	Get specific order	Customer	
PATCH	/orders/{id}/cancel	Order Lambda	Cancel an order	Customer	
GET	/orders/history	Order Lambda	View cancelled/previous orders	Customer	

POST /products
{
 "name": "Wireless Mouse",
 "description": "Ergonomic wireless mouse with USB receiver",
 "category_id": 2,
 "price": 799.00,
 "stock": 50,
 "status": "ACTIVE"
}
PATCH /products/{id}
{
 "price": 699.00,
 "stock": 75
}
GET /products
GET /products
GET /products/{id}
GET /products/101

For Deletion Purpose

PATCH /products/{id}

```
{  
  "status": Not Active,  
}
```

POST /orders

```
{  
  "items": [  
    {  
      "product_id": 101,  
      "quantity": 2  
    },  
    {  
      "product_id": 205,  
      "quantity": 1  
    }  
  ]  
}
```

GET /orders

GET /orders

GET /orders/{id}

GET /orders/{id}

For order Cancellation Purpose

PATCH /orders/{id}/cancel

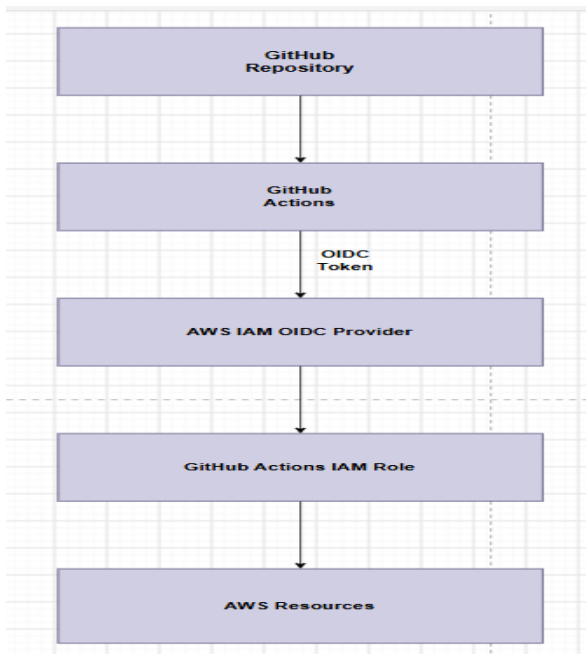
```
{  
  "reason": "Customer requested cancellation"  
}
```

GET /orders/history

GET /orders/history

CI/CD Workflow Step

1. **Trigger:** Push/merge to main.
2. **Checkout & Build:** Checkout code and install dependencies.
3. **Package Lambdas:** Build packages for all Lambda functions.
4. **Deploy Stacks:** Network → Database → Application → Monitoring.
5. **Deploy Code:** Update Lambda packages and configurations.
6. **Smoke Test:** Verify authentication, product APIs, order placement, order retrieval, and stock-threshold event flow.
7. **Authentication:** GitHub Actions uses **OIDC** to securely access AWS.



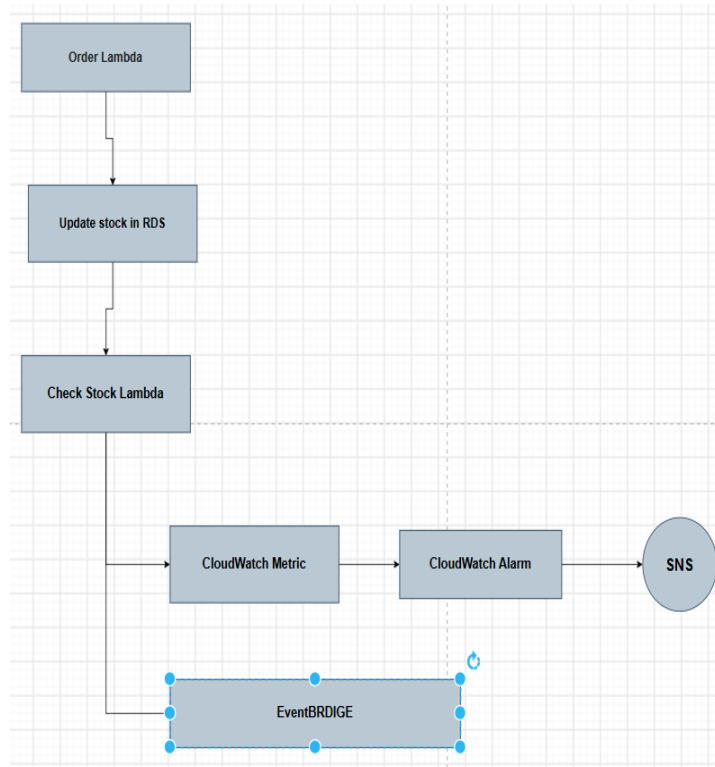
Custom CloudWatch Metrics

1. **Namespace:** CloudMart/Application
2. **Order Lambda:** OrdersPlaced — published after a successful order. Unit: Count.
3. **Order Lambda:** InsufficientStock — published when requested quantity exceeds available stock. Unit: Count.
4. **Check Stock Lambda:** LowStockProducts — published when stock falls below the defined threshold. Unit: Count.
5. **Dimensions:** Environment (dev/prod) and Function (Order/CheckStock).
6. **Publishing point:** Metrics are published only after the corresponding business condition occurs, so CloudWatch reflects actual application events.

Error Handling Strategy

1. **Lambda Errors:** Catch exceptions, log the error details in CloudWatch Logs, and return a consistent error response to the caller.
2. **Client Errors:** Return appropriate 4xx responses for invalid input, unauthorized access, product not found, or insufficient stock.
3. **Server Errors:** Return 500 Internal Server Error for unexpected application or database failures.
4. **DLQ:** Configure a DLQ for asynchronous Lambda processing, mainly for failures in the post-order stock/event flow.
5. **Custom Metrics:** Publish OrderFailures, InsufficientStock, and LowStockProducts metrics to CloudWatch when the respective conditions occur.

Alarms



Event Structure

Order-created event

```
{  
  "source": "cloudmart.orders",  
  "detail-type": "OrderCreated",  
  "detail": {  
    "orderId": "ORD1001",  
    "customerId": "CUS101",  
    "productId": "P101",  
    "quantity": 2,  
    "status": "PLACED"  
  }  
}
```



```
{
  "source": "cloudmart.orders",
  "detail-type": "OrderCancelled",
  "detail": {
    "orderId": "ORD1001",
    "customerId": "CUS101",
    "productId": "P101",
    "quantity": 2,
    "status": "CANCELLED",
    "reason": "CUSTOMER_REQUEST"
  }
}
```

Low-stock event

```
{
  "source": "cloudmart.inventory",
  "detail-type": "LowStockDetected",
  "detail": {
    "productId": "P101",
    "productName": "Laptop",
    "currentQuantity": 8,
    "threshold": 10,
    "eventType": "LOW_STOCK"
  }
}
```